Meshtastic CLI Architecture

Communication to Radio

Level 1: Link connection

This level should define the way the data is transmitted between sender and receiver.

Basic format of data sent (e.g. over serial line):

The transmitted data is separated in a 4 Byte header and the data part.

ByteNo	Value	Comment
1	0x94	START1
2	0xC3	START2
3	LenH	High byte of data length
4	LenL	Low byte of data length
5	Data 0	first data byte
...		
N	Data N	Last data byte

This format is used for both serial and TCP connections to the radio. When using BLE, only the data itself will be transmitted. BLE is taking care about the header (which basically also is true for TCP).

The data must be formatted as a "ToRadio" protobuf structure.

Level 2: ToRadio/FromRadio packets

This level should define some control messages and otherwise feed through different types of packets according to their "envelope".

Control Messages are:

- disconnect
- Heartbeat
- want_config_id (somehow in between, because it delivers a lot of data but it serves otherwise to start communication)
- config_complete_id (denotes the full availability of local radio)
- rebooted

- QueueStatus
- FileInfo

ToRadio: Support 6 different packets:

1. MeshPacket (ID 1): a regular packet sent to the mesh
2. want_config_id (ID 3): requests actual config of local node, using a random ID
3. disconnect (ID4): announces a disconnection
4. XModem packet (ID 5)
5. MqttClientProxyMessage (ID 6)
6. Heartbeat (ID 7): to keep the connection active

FromRadio:

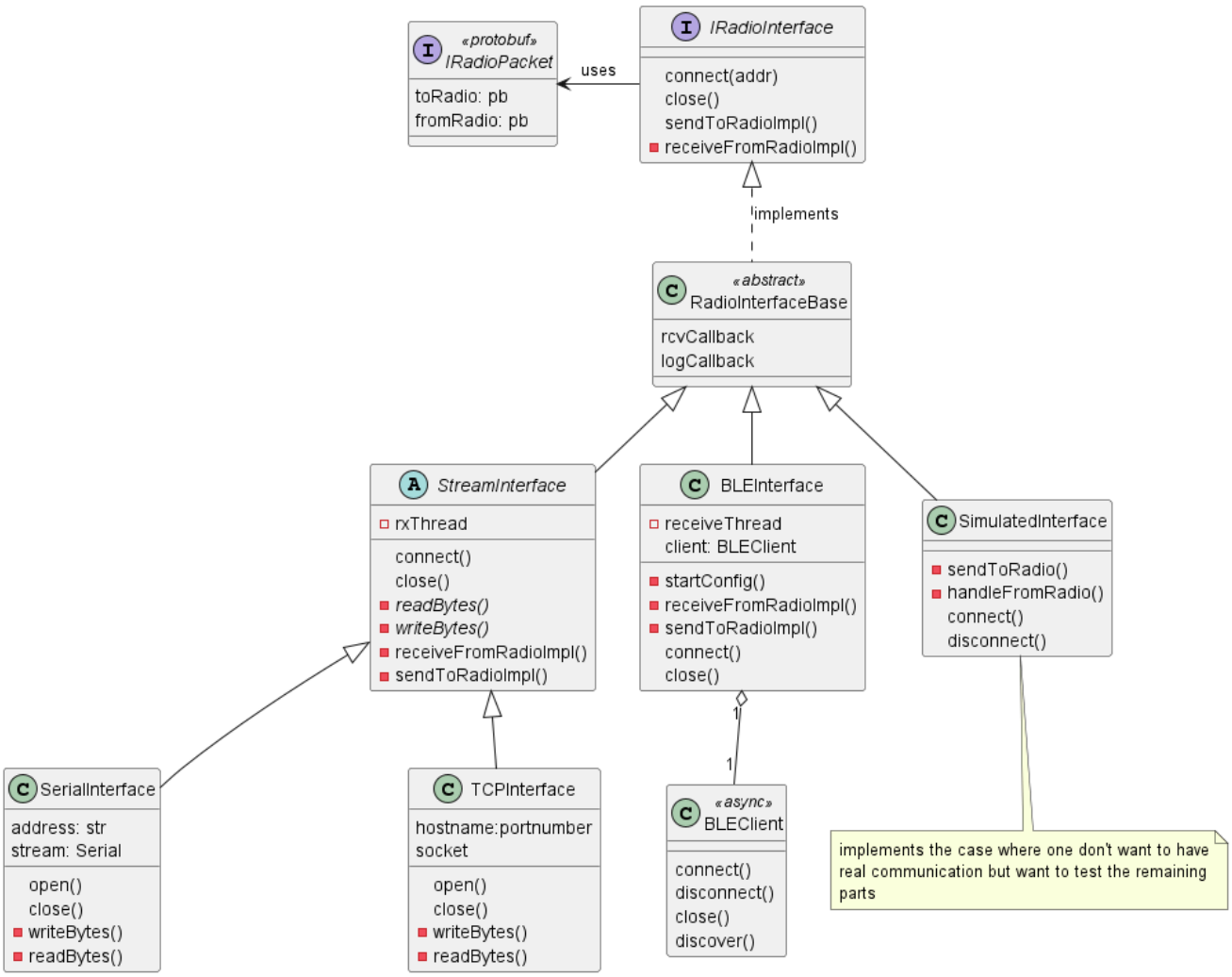
1. packet_id: not clear if this is always filled in and what it exactly means
2. Payload with variants:
    iii. MeshPacket
    iv. MyNodeInfo (partial response to of want_config_id)
    v. NodeInfo (partial response to of want_config_id)
    vi. Config (partial response to of want_config_id)
    vii. LogRecord
    viii. config_complete_id: returns initial sent ID when all config has been sent
    ix. rebooted
    x. ModuleConfig (partial response to of want_config_id)
    xi. Channel (partial response to of want_config_id)
    xii. QueueStatus
    xiii. XModem
    xiv. DeviceMetaData (partial response to of want_config_id)
    xv. MqttClientProxyMessage
    xvi. FileInfo
    xvii. ClientNotification
    xviii. DeviceUIConfig

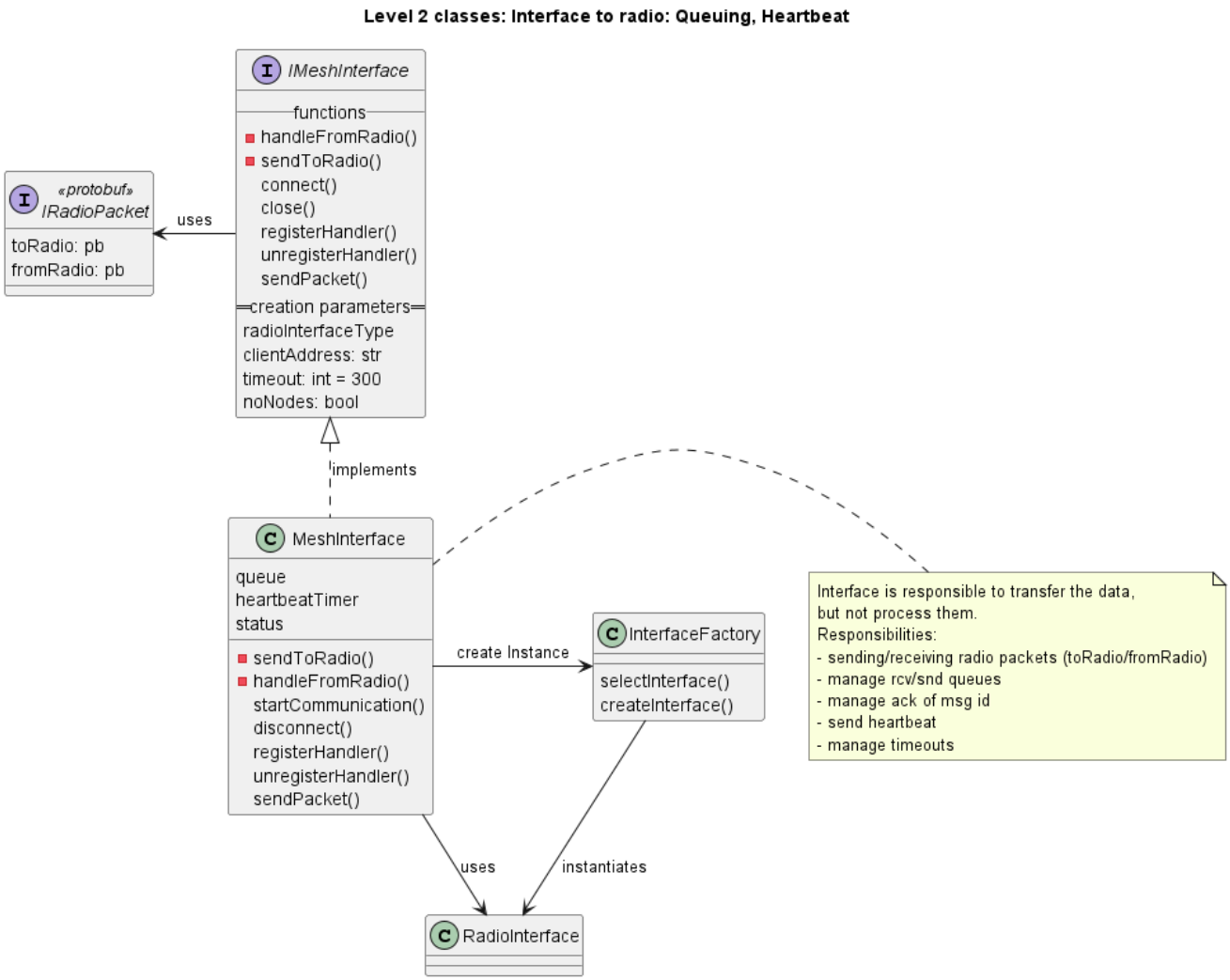
This level should also handle the send/receive queue to ensure that no overflow happens on either side.

This means in particular, that the received QueueStatus messages from the radio are interpreted and used to throttle/limit the flow of messages towards the radio.

On the lowest level exist the direct interfaces to the radios using the different data transmission protocols. Those are defined by the *IRadioInterface*

Level 1 classes: Radio Interface definition





Status handling

Status data contains:

- isConnected: all initial data has been transferred from radio to client
- rebooted: timestamp of last reboot.
- error: Any error code including timestamp

Whenever a change in the status occurs, the new data will be published over pub-sub

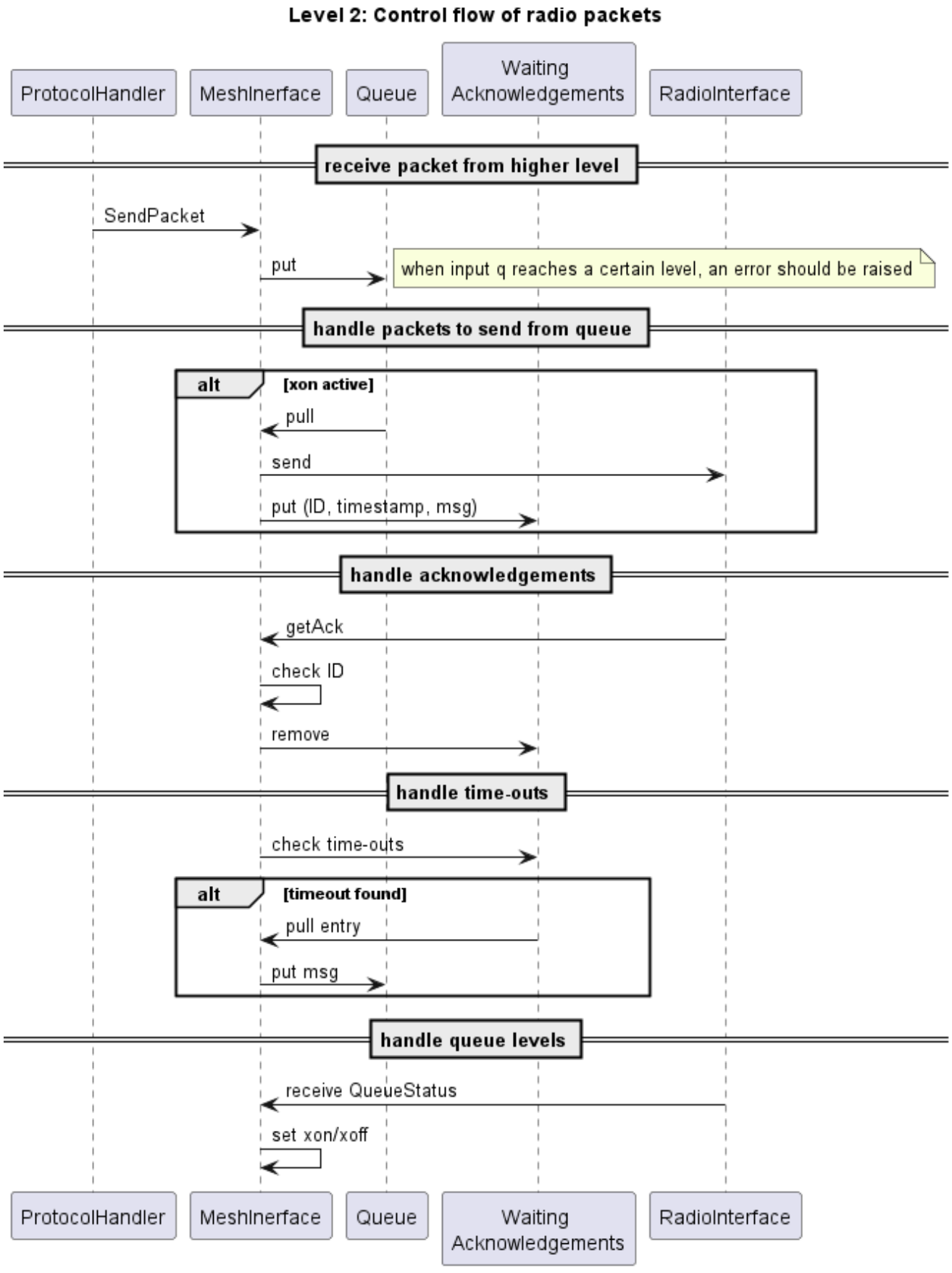
Queue handling

This concerns mainly the transmit queue to prevent overflow of the radio processing queue. The radio transmits the current queue level with a specific QueueStatus packet which then can be used in a XON/XOFF manner to control the flow of data from MeshInterface to the radio. Max queue places: N=16 Stop sending at: xoff = N-2 = 14 Resume sending at: xon = N/2 = 8

Control flow

Mesh packets contain a message ID, which can be used to validate that the message has been processed correctly within the radio (Actually it happens quite often, that messages get lost). A lost package should be resent if no ACK is

receieved within a resend time-out.



Level 3: Higher Level Packets

Those packets are:

- MeshPacket
- XModem
- MqttClientProxyMessage
- LogRecord
- ClientNotification
- Config-Data*
- ModuelConfig-Data*
- Channel-Data*
- initial config data: MyNodeInfo, NodeInfo, DeviceMetaData, DeviceUIConfig

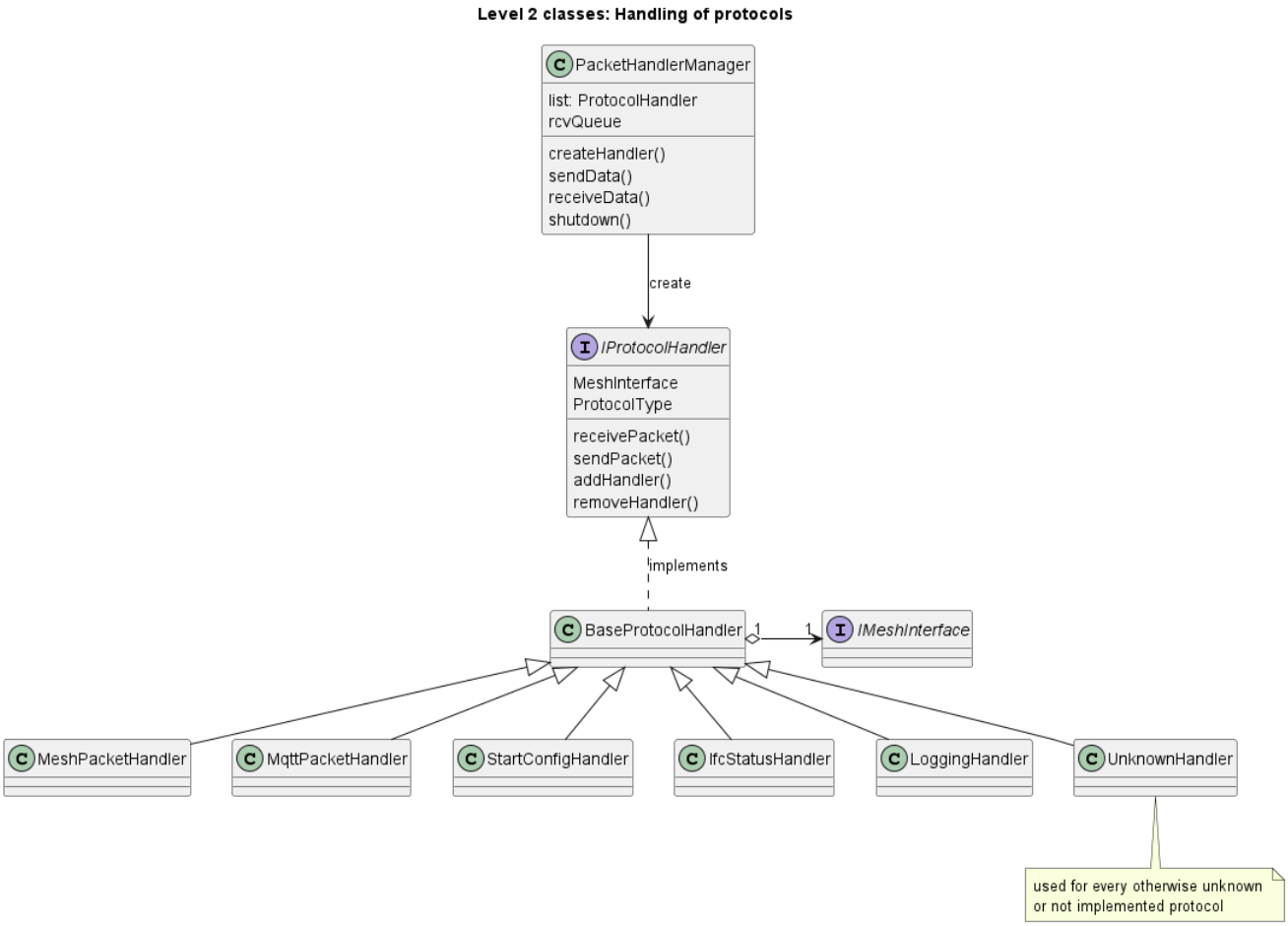
*) Also received during initial config transmission

Those packets need to be decoded according to their content by the appropriate handler Handlers should register themselves at the receiver, so they can called when the respective packet arrives.

Implementation note: Receiving data is currently executed in a separate receiving thread. It seems to be useful to keep decoding data in the protocol handlers within this thread, but switch back to the main thread when publishing the decoded data.

The idea here is, that the handlers queue the decoded data in the manager and set a flag. This will waken the main thread to take the data and publish it to the subscribed applications.

Might be useful to use async functions here.



Level 4: Applications

Those applications are:

- start communication and receive initial data
- CLI commands
- Logging data
- MQTT messages
- XModem
- other (Client Notification?)

In order to realize the communication between applications (which might be there or not) and the protocol handlers, a Pub-Sub Model seems to be appropriate.

Using this model, any application can subscribe whenever it is active and then receives the data it will use. Thus, the application will not need to know much about the underlying capabilities: if something is missing, an error will be returned.

Pub-Sub Topics

Following topics will be defined:

Root topic: "meshtastic" Child Topics: according to the applications discovered:

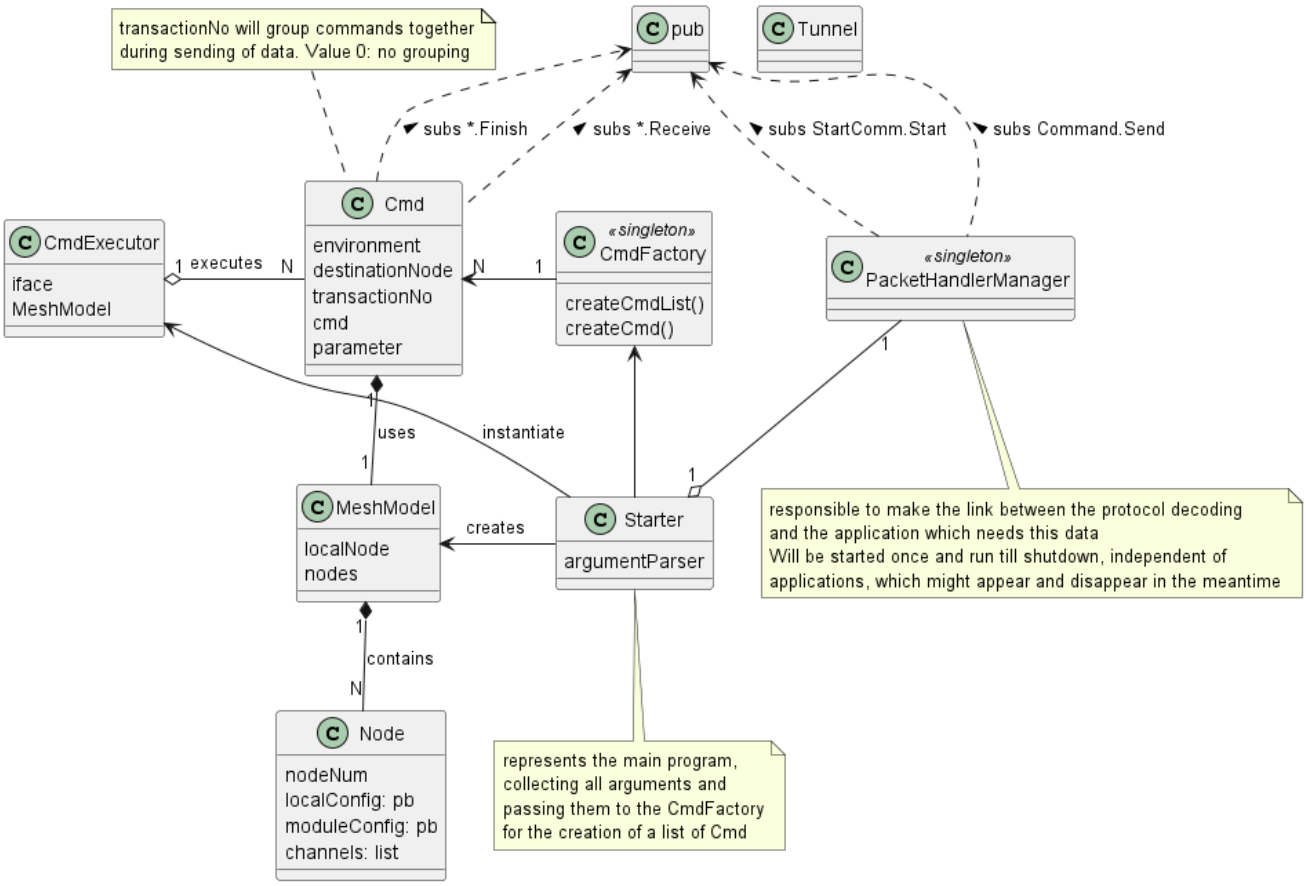
- "Logging"
- "MQTT"
- "XModem"
- "ClientNotification"
- "StartCommunication"
- "Command"

SubTopics of these are application specific:

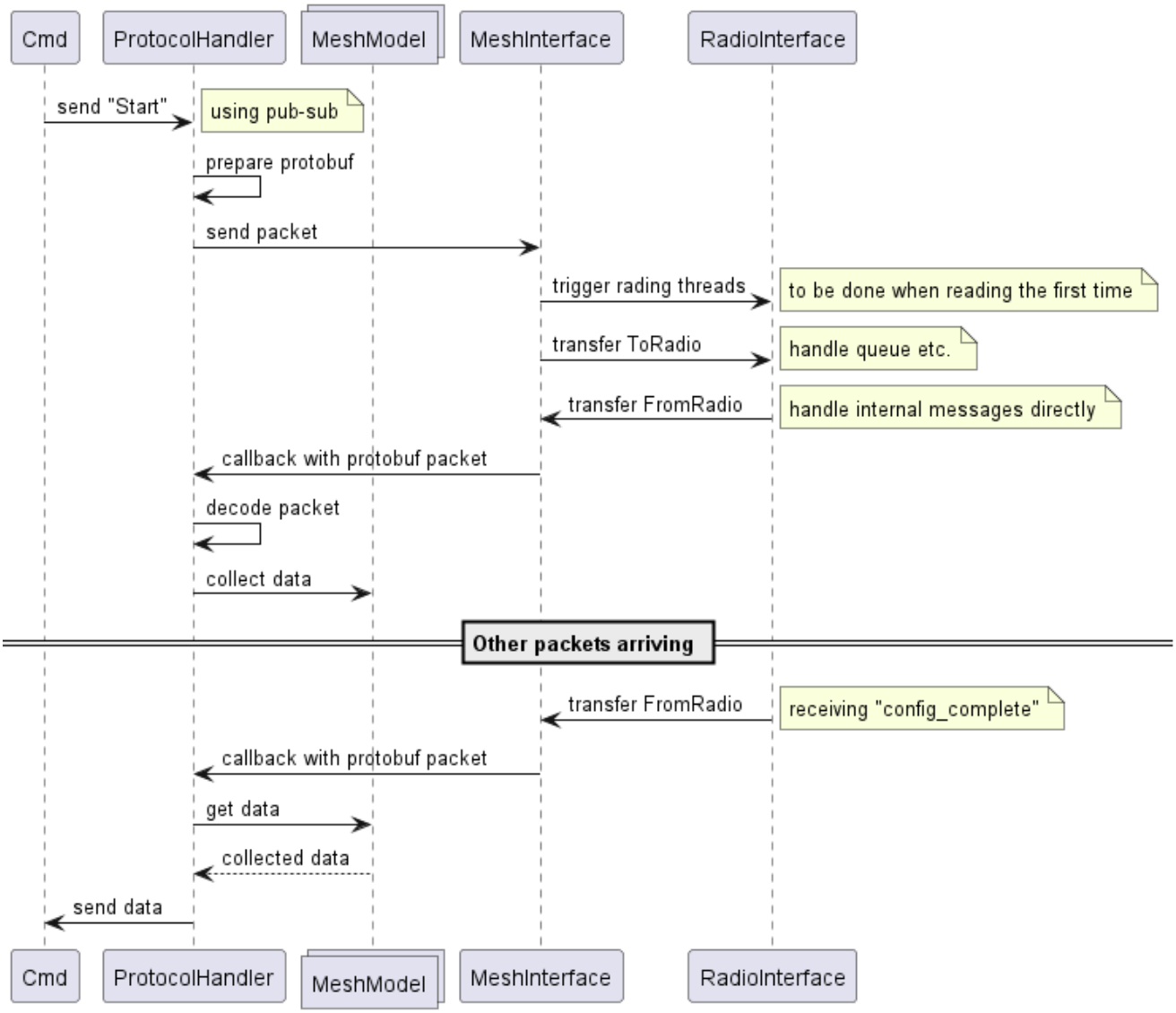
Child topic	Sub topic	Comment
Logging	Receive	Logging app will receive a new entry
StartCommunication	Start	Trigger communication start
	Receive	a new data part is received, the data will explain itself
	Finish	all data received, app can be terminated
Command	Send	send a request from application to protocol handler
	Receive	receive a response, this might be only part ot the whole data
	Finish	all packets received, command can be terminated

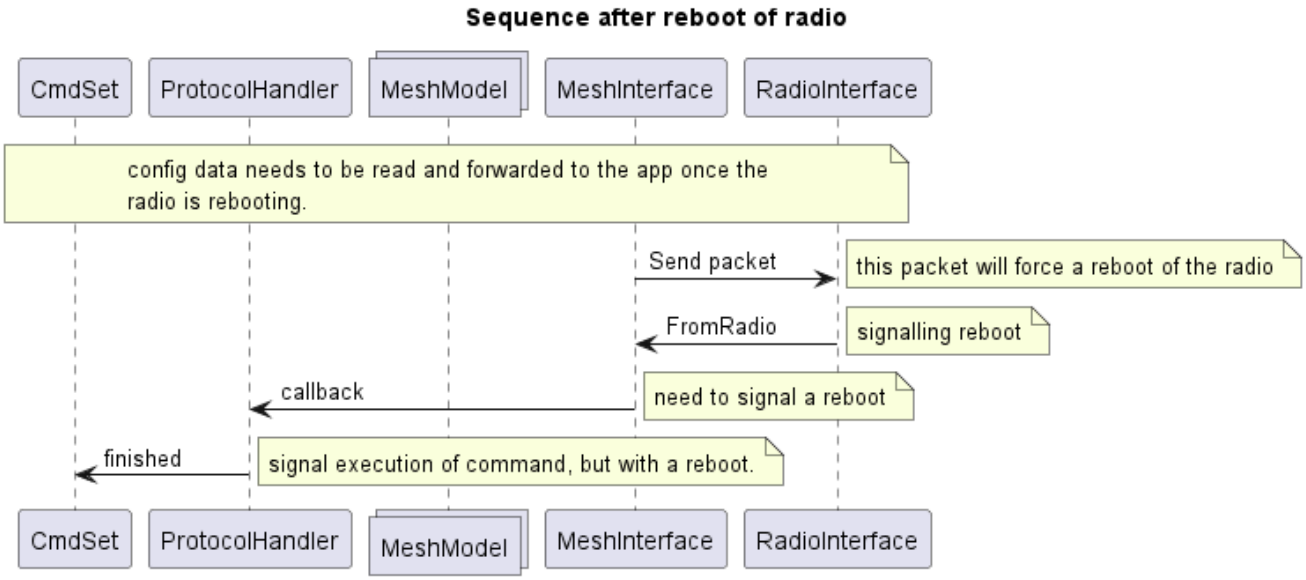
Callables shall be named with the name of the Sub-topic, adding a prefix "on", e.g. "onReceive"

Level 3 classes: Application "Command"



Sequence for initiating reading initial configuration via a cmd




Sequence after reboot of radio
CmdSet
ProtocolHandler
MeshModel
MeshInterface
RadioInterface
config data needs to be read and forwarded to the app once the radio is rebooting.
Send packet
this packet will force a reboot of the radio
FromRadio
signalling reboot
callback
need to signal a reboot
finished
signal execution of command, but with a reboot.
CmdSet
ProtocolHandler
MeshModel
MeshInterface
RadioInterface